NAME- YUVIA

Debugging of faulty sensor

Start
↓
Check Power supply
↓
Power OK ? → No → Fix supply
Yes ↓
Check wiring
↓
Loose? → Yes → Repair wiring
No ↓
Replace sensor
↓
New sensor working? → Yes → Old sensor was faulty
↓ No
Test ADC
↓
ADC correct? → Yes → debug software
↓ No
Calibrate ADC

Ans. Hardware modification:-

→ use Li-ion Battery ⇒ Longer Backup
→ use solar panel ⇒ continuous charging

BATCH- IoT & DRONES

→ use low power sensor → lower energy consumption
→ Disconnect unused peripherals → reduces leakage current

Software modification:-

→ apply deep sleep mode → current drops to a few microamps
→ Read sensor every 15-20 min → less CPU activity
→ upload data in batches → fewer Wi-Fi connections
Optimize code → Faster execution & lower energy use.

Ans. Automatic Braking communication

CAN Bus
↳ controlled area Network

Reason:- ① Very low latency
② Highly Reliable
③ error detection
④ used in automobiles
⑤ works even under electrical noise.

Wi-Fi or Bluetooth are unsuitable because of unpredicted delay

Ans. The sensor should be placed:

→ At the center of the classroom
→ At approx half the room height (h/2)
→ away from windows, doors, fans, air conditioners, heaters, and direct sunlight

(a/2), (a/2), (a/2)

This location provides the most representative average temp because it minimizes the influence of localized heat sources or airflow

ceiling
(a/2) (a/2, b/2) sensor
floor

Effect of Poor Placement:-

① Near window → Higher temp due to sunlight
② Near AC → Lower than actual room temp.

(2) Near door → fluctuating readings
(4) Near ceiling → Hot air accumulates higher readings
(5) Near floor → cooler air settles giving lower readings
(6) Near processor / equipment → Artificially high temp.

Ans Analog sensors perform better when
(1) Very high response required
(2) Continuous measurements needed.
(3) Low - cost application.
(4) Simple circuits
(5) High - resolution ADC available
e.g → Potentiometer, LDR, Gas Sensor

(9) Possible Attacks:-
(1) ~~Ata~~ Unauthorised Device Control → Turns relay / pump ON or OFF
(2) Data Theft → Reads sensor information
(3) Data Modification → sends fake readings
(4) Malware Installation → Install malicious firmware
(5) Password Theft → Captures Wi-Fi credentials
(6) Denial of Service (DoS) → Prevents normal operation

Security Improvement:-
(1) WPA2/WPA3 Encryption :- Secures Wi-Fi communication
(2) HTTP / TLS Encryption :- Encrypt transmitted data
(3) Authentication Tokens :- only authorized users access device
(4) Firewall Rules :- Block suspicious traffic
(5) Regular Firmware Updates :- Fix security vulnerabilities.

10. Conflict that can occur:-

| Component | Possible Conflict | Solution |
| --- | --- | --- |
| (1) DHT 2 | Time consuming communication | use dedicated library |
| (2) Ultrasonic Sensor | Blocking delays | uses timers / non-blocking code |
| (3) OLED Display | $I^2C$ address conflict | verify device address |
| (4) Relay Module | Current spikes | use transistor & flyback diode |
| (5) Soil-Moisture Sensor | ADC noise | Avg multiple reading |

11. To prevent data loss:-
(1) Store sensor reading in local memory when the internet is unavailable.
(2) Keep checking the Internet connection.
(3) When the connection is restored upload all the stored data to the cloud in the correct order.
→ After successful upload, delete the local memory.

12. To improve long-term reliability:-
(1) Use durable hardware, good-quality sensors, and a backup power source
(2) Add watchdog timer to restart the device if it hangs.
(3) Use error detection & automatic recovery in software.
(4) Store imp data in non-volatile memory

(13) Yes, An IoT system can be considered smart even without AI. It can automatically perform tasks using sensors, predefined rules, & automation.

e.g → A smart irrigation system turns on the water pump when soil moisture is low, without using AI.

Diff:- Smart:- works on fixed rules & automation
Intelligent:- uses AI/ML to learn from data, adapt to new situations and make better decisions over time.

(14) Polling means the processor continuously checks the status of a device or sensor. It is simple but waste CPU time & power.

Interrupt based programming means the processor performs other tasks & responds only when an event occurs. It is faster & more efficient.

Interrupt improves IoT performance in situation like motion detection, emergency alarms, button press.

(15) In my smart water system, I made the following trade offs:

1. I used ESP 32, which increased cost slightly but provided WiFi connectivity.
2. I uploaded data every min instead of continuously to save power & internet data.
3. I used basic sensors, which reduced cost but gave slightly lower accuracy.

16

Start
↓
Initialize ESP 32 & Sensors
↓
Read sensors Data
↓
WiFi Available?

Yes → Upload Data to cloud
No → Store Data in Flash / SD Card

↓
Check Wi-Fi again
↓
If connected, upload stored Data
↓
END

→ Prototype: 1. Program: Arduino IDE (ESP32 code)
2. Protocol: WiFi, HTTP/MQTT
3. Input (I): Temp / Water level sensors
4. Output (O): Cloud server, LCD / Buzzer
5. Storage: Flash Memory / SD card

(17)

Start
↓
Connect to Wi-Fi
↓
Check for OTA update
↓
Update Available?

No → Continue Normal work
Yes → Download firmware → Verify firmware → Upload Successful?

Yes → Restart ESP32 & Run
No → Show error / Retry or Rollback

↓
END.

5

Temp Sensor + Water Level Sensor
↓
ESP 32
↓
WiFi / Internet
↓
Cloud Database
↓
Mobile App / Web Dashboard

| Failure Point | Solution |
| --- | --- |
| 1. Sensor failure | Replace / calibrate sensor |
| 2. ESP 32 hangs | use watch dog timer |
| 3. Wi-fi failure | Store data in flash |
| 4. Cloud server down | Retry upload automatically |
| 5. Power failure | Use battery backup / UPS |
| 6. User interface not loading | Refresh app & check internet |

→ it can easily read the soil moisture sensor
→ it can control relay
→ lower cost and lower power consumption
→ the program is simple and requires little memory

Ans.

| Features | Wi-Fi | Bluetooth | LoRa | MQTT |
|---|---|---|---|---|
| Range | 50-100 m | 10-100 m | 2-15 km | depends on network |
| Data rate | very High | Medium | Low | lightweight |
| Power consumption | High | Low | very low | very low |
| Internet support | Yes | No | through gateway | Yes |
| No. of devices | moderate | limited | Thousands | Thousands |
| Best use | High-speed data | wearables | Long-range sensor | cloud communication |

For 500 sensors distributed across a 2km area, the protocol or combination of protocols that can be chosen are LoRa and MQTT.

Benefits of LoRa:

→ covers entire 2km factory
→ supports hundreds of sensors
→ very low battery consumption
→ Reliable in industrial environment.

__Benefits of MQTT.__

→ Lightweight communication protocol
→ efficient cloud communication
→ supports Publish - Subscribe architecture
→ Reliable even with unstable networks

The combination of LoRa for long range sensor communication and MQTT for cloud communication provides the smart scalable, reliable, and energy - efficient solution.

__Ans3__ To determine whether the problem lies in the sensors, wiring, power supply, ADC or software, always isolate one component at a time instead of changing everything simultaneously.

__Eg:__ If it is a faulty sensor, it will show incorrect readings constantly.
If it shows wrong analog values, there is an ADC problem.

Que You are building a smart irrigation system ... was not required

Ans I would choose the ESP 32 because it to has built-in wi-fi & bluetooth. making it easy to send sensor data to a mobile application every 5 sec. it is also much faster than the Arduino UNO. has more memory, and supports multitasking

To monitor soil moisture, control a water pump, and send data to a mobile application every 5 sec. we need ESP32 because:
it has built-in wi-fi which enables direct communication with the mobile application
faster processor allows simultaneous sensor reading, relay control, and cloud communication.
Bluetooth can be used for future expansion.
More RAM & flash support longer program

if internet is not required, then Arduino UNO is a suitable and economic option because :